

Iteration

1. The for loop

Iterate over a string with for

“Iterate over a string”, or “loop over a string”

Loop over "abcd", with the loop variable letter.

letter receives a new value in every iteration.

```
1 alphabet = "abcd"
2
3 nr = 0
4 for letter in alphabet:
5     print(str(nr) + ":" + letter)
6     nr = nr + 1
```

0:a

1:b

2:c

3:d

Iterate over a list with for

“Iterate over a list”, or “loop over a list”

Loop over ["a", "b", "c", "d"], with the loop variable e1.

e1 receives a new value in every iteration.

```
1 letters = ["a", "b", "c", "d"]
2
3 nr = 0
4 for el in letters:
5     print(str(nr) + ":" + el)
6     nr = nr + 1
```

0:a

1:b

2:c

3:d

Iterate over a tuple with for

“Iterate over a tuple”, or “loop over a tuple”

Loop over ["a", "b", "c", "d"], with the loop variable e1.

e1 receives a new value in every iteration.

```
1 letters = ("a", "b", "c", "d")
2
3 nr = 0
4 for el in letters:
5     print(str(nr) + ":" + el)
6     nr = nr + 1
```

0:a

1:b

2:c

3:d

Iterate over a list with for

Since a list is mutable, it is **technically** possible to change it's content while you loop over it.

However, never change the content while lopping over it. Will lead to confusing behaviour.

```
1 letters = ["a", "b", "c", "d"]
2
3 nr = 0
4 for el in letters:
5     letters.pop(0) # DON'T DO THIS IN A LOOP!!!
6     print(str(nr) + ":" + el)
7     nr = nr + 1
```

0:a

1:c

2. Save the results from the loop

In a typical use case, you often want to save some kind of result from processing conducted in the loop. For instance, in a list.

Note that this list is defined **outside** of the loop!

```
1 def get_ing_words(tokenized_sentence):
2     ing_words = []
3     for token in tokenized_sentence:
4         if token.endswith("ing"):
5             ing_words.append(token)
6     return ing_words
7
8 words = ["they", "were", "singing", ":",
9         "fish", "were", "jumping", "and",
10        "the", "cotton", "was", "high"]
11
12 result = get_ing_words(words)
13 print(result)
```

['singing', 'jumping']

Also when you loop over a string, it is in many cases practical to save the result in a list.

```
1 ENGLISH_VOWELS = ["a", "e", "i", "o", "y"]
2 def get_vowels_from(text):
3     found_vowels = []
4     for ch in text:
5         if ch.lower() in ENGLISH_VOWELS:
6             found_vowels.append(ch)
7     return found_vowels
8
9 greeting = "Good morning, fellow being! All is good!"
10
11 all_vowels = get_vowels_from(greeting)
12 print(all_vowels)
```

['o', 'o', 'o', 'i', 'e', 'o', 'e', 'i', 'A', 'i', 'o', 'o']

3. The leaking loop variable

You can still access the loop variable (here the variable called `el`) after the loop is finished.

But it makes no sense, you very rarely need this.

```
1 letters = ("a", "b", "c", "d")
2
3 nr = 0
4 for el in letters:
5     print(str(nr) + ":" + el)
6     nr = nr + 1
7
8 print("el value in last iteration: " + el)
```

0:a

1:b

2:c

3:d

el value in last iteration: d

4. Range

Range

Introducing a new built-in type:

`range`

(and actually also a new built-in function: `range()`)

```
1 numbers = range(0,10)
2
3 print(numbers)
4 print(type(numbers))
```

```
range(0, 10)
```

```
<class 'range'>
```

Range

Transforming it to a list can make us understand what it is.

```
1 numbers = range(0,10)
2
3 print(list(numbers))
```

[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

Range

Often used in loops, when you need a number that increases.

```
1 for i in range(0, 5):  
2     print(i)
```

0

1

2

3

4

Often used in loops, when you need a number that increases.

```
1 # A not very efficient implementation
2 def get_words_of_len(words, word_len):
3     return_words = []
4     for word in words:
5         if len(word) == word_len:
6             return_words.append(word)
7     return return_words
8
9 song_words = ["we", "were", "singing", ":", "fish", "were",
10              "jumping", "and", "the", "cotton", "was", "high"]
11
12 for i in range(1, 8):
13     print(get_words_of_len(song_words, i))
```

```
[':']
['we']
['and', 'the', 'was']
['were', 'fish', 'were', 'high']
[]
['cotton']
['singing', 'jumping']
```

You can also decide how large a step to take

```
1 for i in range(0, 10, 2):  
2     print(i)
```

0

2

4

6

8

5. Breaking a loop

With break, you can break a loop:

```
1 def print_first_city(tokenized_sentence):
2     cities = ("Stockholm", "Oslo", "London", "Bergen")
3     for word in tokenized_sentence:
4         if word in cities:
5             print(word)
6             break
7
8 s = ["I", "am", "going", "to", "Stockholm", "Oslo",
9     "and", "Bergen"]
10
11 print_first_city(s)
```

Stockholm

To write return in the middle of a loop also results in a kind of break.

```
1 def get_first_city(tokenized_sentence):
2     cities = ("Stockholm", "Oslo", "London", "Bergen")
3     for word in tokenized_sentence:
4         if word in cities:
5             return word
6
7 s_1 = ["I", "am", "going", "to", "Stockholm", "Oslo",
8       "and", "Bergen"]
9 s_2 = ["Hello", "there", "world", "?"]
10
11 print(get_first_city(s_1))
12 print(get_first_city(s_2))
```

Stockholm

None

6. while

To write return in the middle of a loop also results in a kind of break.

```
1 # Collect three words containing e
2 words_with_e = []
3 while len(words_with_e) < 3:
4     user_wprd = input('Write a word: ')
5     if "e" in user_wprd.lower():
6         words_with_e.append(user_wprd)
7
8 print('The words are:', words_with_e)
```

Write a word: hello

Write a word: hi

Write a word: e

Write a word: bye

The words are: ['hello', 'e', 'bye']